

Study Planner Agent: A Retrieval-Augmented Course Review System for Topic-Centered Study Planning

Shengxiang Qi

University of Illinois Urbana-Champaign

sqi8@illinois.edu

ABSTRACT

Students often accumulate lecture slides, notes, and course documents without having a structured way to turn those materials into an actionable study workflow. Generic large language models can explain concepts, but they are not course-grounded and do not reliably organize uploaded materials into a personalized review process. This project presents *Study Planner Agent*, a web-based development-track system that supports course-specific document upload, chunk-based retrieval, topic extraction, study plan generation, topic-by-topic review, and grounded course chat in one integrated workspace. The system uses a FastAPI backend, a React frontend, PostgreSQL for relational storage, and Milvus for vector retrieval. Beyond software implementation, the project includes an evaluation framework that measures both retrieval quality and end-to-end answer quality against a plain general LLM baseline. Results show that the system consistently outperforms a plain LLM on groundedness and course specificity, while also revealing that retrieval quality—especially keyword retrieval on natural-language questions—remains the main bottleneck on real course materials. The project demonstrates that a retrieval-grounded study assistant can provide more useful course-specific support than a generic conversational model while still leaving clear opportunities for improving retrieval robustness.

KEYWORDS

retrieval-augmented generation, study planning, topic extraction, course chat, development track

1 INTRODUCTION

Students preparing for exams often have plenty of source material but no reliable system for transforming that material into a coherent study process. Lecture slides, PDFs, notes, and example problems may all exist, yet students still need to answer several practical questions: What are the major topics in this course? What should be reviewed first? Which documents support each topic? Can a system answer course questions without drifting away from the uploaded material?

This project addresses that problem by building a course-grounded study assistant rather than a general-purpose chatbot. The resulting system, *Study Planner Agent*, lets a student create a course workspace, upload documents, automatically extract course topics, generate a study plan, review topics with supporting chunks, and ask grounded questions over the course corpus. The project is a solo development-track effort whose main deliverable is the working software system and its supporting documentation.

The key idea is that a useful study assistant should operate over the student's own course materials. Instead of answering from generic background knowledge alone, the system should retrieve relevant

chunks from uploaded documents, attach them to topics, and use them to support study-plan generation and course question answering. This makes the tool more suitable for course review, where correctness alone is not enough; answers should also be tied to the actual course content.

2 MOTIVATION AND GOALS

The motivation for this work is the gap between document storage and actual study support. Existing tools often do one of the following well but not all at once: store documents, support semantic search, answer questions, or provide planning. In practice, students need a workflow that connects these functions. The intended user is a student who wants to upload course material and convert it into a structured review process.

Another motivation is a limitation of current large language model usage in study settings. Even when modern models provide large context windows, a student cannot simply paste an entire semester of lecture slides, notes, and assignments into one prompt and expect consistently precise answers. Large mixed-context prompts often dilute the most relevant evidence, increase noise, and make it harder for the model to focus on the exact portion of the course material that matters for a question. A course-grounded retrieval system is therefore useful not only because it adds sources, but also because it controls context selection.

There is also a workflow limitation in plain general LLMs. A general model can answer a question, but it does not inherently track whether the student has already reviewed a topic, whether a topic still needs work, or what the next step in a study plan should be. In other words, it can produce explanations, but it does not naturally turn a collection of class materials into an organized review loop. This project is motivated by that gap between answering and supporting actual learning progress.

The project goals were:

- Build a usable web-based course workspace.
- Support PDF and DOCX ingestion with asynchronous processing.
- Extract study topics from uploaded materials.
- Generate a study plan from those topics.
- Provide topic-centered review and grounded course chat.
- Evaluate both retrieval quality and end-to-end answer quality.

The resulting system is not intended to replace a foundation model. Instead, it is meant to improve the usefulness of that model in the specific setting of course review by grounding it in user-uploaded course content.

3 SYSTEM OVERVIEW

3.1 Core Workflow

The end-to-end workflow is:

- (1) A student creates a course.
- (2) The student uploads PDF or DOCX course materials.
- (3) The backend parses and chunks the document in the background.
- (4) Child chunks are indexed for retrieval and parent/child metadata is persisted.
- (5) Topics are extracted and linked to source chunks.
- (6) The student generates a study plan from active topics.
- (7) The student reviews topics, asks grounded questions, and tracks progress.

3.2 Architecture

The software stack is:

- **Frontend:** React, TypeScript, Vite
- **Backend:** FastAPI, Python 3.12
- **Relational data:** PostgreSQL
- **Vector retrieval:** Milvus
- **LLM interface:** OpenAI-compatible chat API

The backend is organized into modular routers and services. Major subsystems include authentication, course management, document ingestion, chunking, retrieval, topic extraction, study plan generation, and chat. This separation was important because the system evolved in stages, and later features such as topic review and study-plan execution depend directly on earlier ingestion and retrieval outputs.

4 MAJOR SYSTEM FUNCTIONS

4.1 Course Workspace and Document Ingestion

The workspace is the entry point for the user. It presents course materials, processing state, and navigation to topics, study plans, search, and chat. Uploaded documents are processed asynchronously so that the frontend can respond quickly while the backend performs parsing, chunking, indexing, and topic extraction in the background.

This part of the system matters because the project is not only a question-answering interface. It is a persistent course workspace. The user needs to know which documents belong to the course, which files are still being processed, which files have already produced usable chunks and topics, and where to go next in the workflow. The workspace therefore acts as an operational hub rather than just a landing page.

The ingestion pipeline performs the following steps:

- (1) Save the uploaded file.
- (2) Create a processing job record.
- (3) Parse the document text.
- (4) Build hierarchical chunks.
- (5) Index child chunks for retrieval.
- (6) Extract or synchronize course topics.

The job-tracking design is important for usability because document processing is one of the longest operations in the system. Without explicit processing stages, the software would feel unreliable whenever uploads took more than a few seconds. By exposing

document status and refreshing the UI around asynchronous jobs, the system makes a slow backend pipeline feel understandable and manageable from the user's point of view.

4.2 Topic Extraction and Topic Review

After chunking and indexing, the system extracts course topics and links them to source chunks. Topics are persisted in PostgreSQL with metadata such as description, keywords, importance, difficulty, and mastery status. A separate topic review view allows the student to inspect one topic at a time, see supporting chunks, and ask a topic-scoped question.

The topic review page is where the system shifts from organization to learning. Instead of only listing document chunks, it provides a topic-level unit of study that can be reviewed, questioned, and tracked. This is important because a useful study assistant should expose a semantic structure of a course, not just a search box.

This function is also where retrieval and planning become more interpretable. A retrieved chunk by itself is often too low-level for a student who is trying to answer "What should I review today?" A topic gives that chunk a learning-oriented label and bundles it with related supporting material. The review page then becomes a stable place where the user can read a short topic summary, inspect the source evidence, ask a focused question, and update mastery state without having to reconstruct context manually.

4.3 Study Plan Generation and Execution

Study plans are generated from active topics and stored as ordered plan items. Each item includes focus points, notes, context snippets, effort estimates, and execution state. The system also supports updating topic mastery and plan-item status so the study plan can serve as an actual execution interface rather than a static list.

This design supports a central development-track goal: move from information storage to actionable study workflow. A study plan item is linked back to a topic, and a topic links back to supporting chunks, so the user can move from plan to content to explanation in a consistent loop.

The execution support is intentionally lightweight but important. Marking a plan item as started or completed, and marking a topic as reviewing or mastered, gives the software a notion of progress that a plain chatbot does not have. This is one of the clearest differences between a study system and a generic conversational interface: the software is not only responding to isolated queries, but also maintaining a persistent record of where the student is in the review process.

4.4 Grounded Search and Course Chat

The system also supports search and course chat over uploaded materials. Search can run in keyword, vector, or hybrid mode. Chat uses retrieval results to produce sourced answers, and the interface displays the answer together with source snippets.

This is a major distinction between the project and a plain LLM baseline. A general model can explain concepts, but it does not know which answer is best supported by the student's actual course

Study Planner Agent: A Retrieval-Augmented Course Review System for Topic-Centered Study Planning

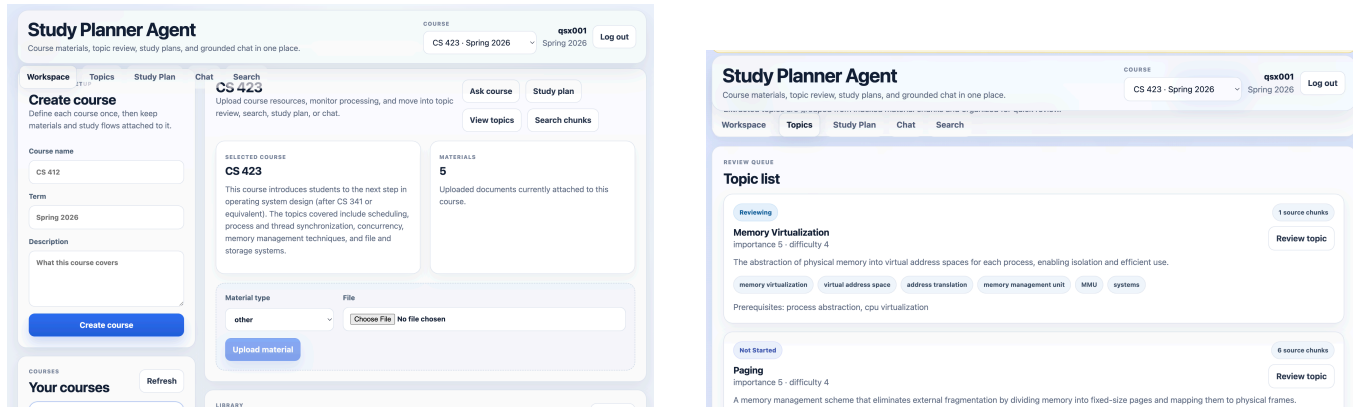


Figure 1: The workspace (left) serves as the operational hub for uploaded course materials and navigation, while the topics page (right) displays extracted topics, topic metadata, and source-backed study concepts.

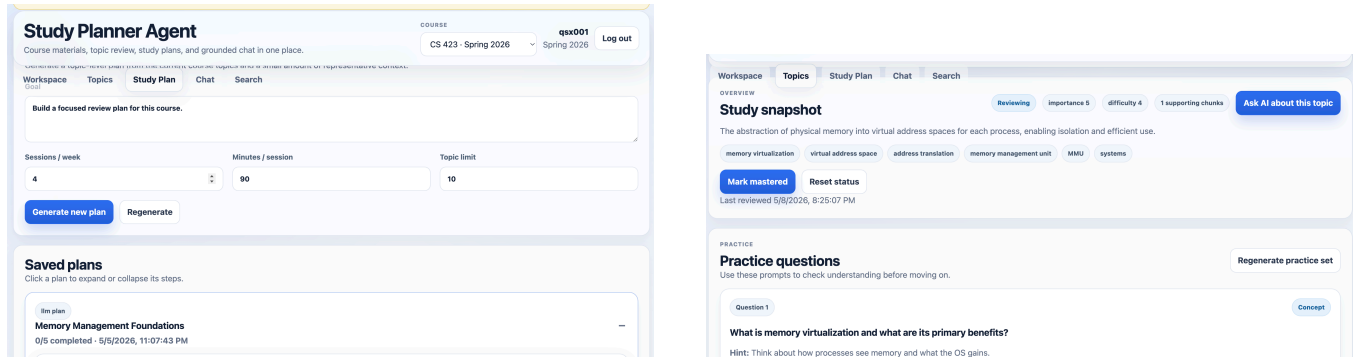


Figure 2: The study plan page (left) turns extracted topics into ordered review tasks, while the topic review page (right) supports focused study with source chunks, grounded Q&A, and mastery tracking.

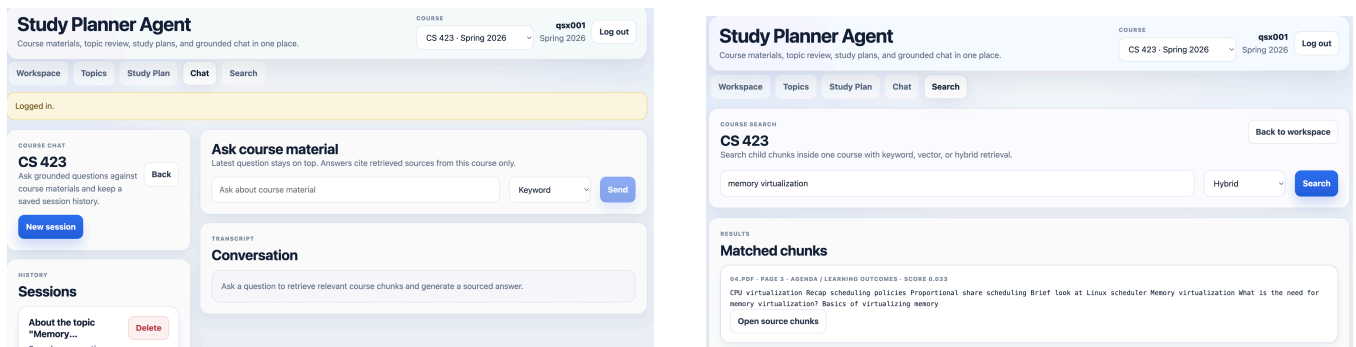


Figure 3: The chat interface (left) provides grounded course answers with visible sources, and the search interface (right) exposes matched chunks under keyword, vector, or hybrid retrieval.

material. The project aims to close that gap by exposing retrieved course evidence directly in the workflow.

Separating search from chat is also a useful product decision. Search gives the user a retrieval-centric interface for manually inspecting matched chunks, while chat gives an answer-centric interface that synthesizes those results. In practice, students often

need both modes: sometimes they want a direct explanation, and sometimes they want to verify or browse the underlying material themselves. Supporting both interaction styles makes the system more usable than a single monolithic chat box.

5 IMPLEMENTATION DETAILS

5.1 Hierarchical Chunking

The backend uses hierarchical chunking with parent and child chunks. Parent chunks preserve larger section context, while child chunks serve as the retrieval unit. This design balances retrieval precision and answer quality. Retrieving only large sections would reduce precision; retrieving only tiny snippets would reduce context quality.

5.2 Retrieval Layer

The retrieval layer supports three modes:

- **keyword retrieval** over PostgreSQL text search
- **vector retrieval** over Milvus embeddings
- **hybrid retrieval** using reciprocal-rank fusion over keyword and vector results

This design made it possible to evaluate retrieval behavior explicitly rather than treating retrieval as a black box. In practice, this also exposed one of the system’s main weaknesses: keyword retrieval is much less effective on natural-language questions than on short term queries.

5.3 Topic Pipeline

The topic pipeline was refactored away from full-course rebuilding and toward an incremental document-to-topic synchronization process. Document-level topic candidates are extracted from parsed sections, then aggregated into course-level topics with source-chunk linkage. Additional curation logic reduces obvious low-quality topics and supports topic status tracking.

This refactor was important because the earlier approach was too slow during upload and deletion. The incremental design reduces unnecessary recomputation and improves stability for downstream features such as study planning.

5.4 LLM Usage

The system can use an OpenAI-compatible chat provider both for grounded chat and for optional topic extraction or curation. However, the software is also designed with fallbacks so that core behavior still exists when an external API is unavailable. This is especially useful for smoke testing and local development.

6 HOW TO USE THE SOFTWARE

The software is intended to be runnable by another developer or evaluator from the repository. The code is available at:

https://github.com/laiwuqxsx/Study-Planner-RAG_Based-Agent

To use the system locally:

- (1) Start PostgreSQL and Milvus with Docker Compose.
- (2) Create backend and frontend environment files.
- (3) Start the FastAPI backend.
- (4) Start the Vite frontend.
- (5) Open the frontend and create a course.
- (6) Upload course materials and wait for processing.
- (7) Use Topics, Study Plan, Review, Search, and Chat from the navigation bar.

Table 1: Round 1 retrieval results on the synthetic OS benchmark.

Mode	Hit@5	Recall@5	MRR
Keyword	0.00	0.00	0.00
Vector	1.00	1.00	1.00
Hybrid	1.00	1.00	1.00

Table 2: Round 1 agent vs. plain LLM results on the synthetic OS benchmark.

System	Corr.	Ground.	Course	Useful
Agent	4.75	4.75	4.75	4.75
Plain LLM	4.75	1.00	1.00	4.25

The repository README contains setup details, environment variables, service URLs, and a recommended demo flow. In addition, the project includes smoke-test scripts for multiple implementation phases, plus an evaluation harness for retrieval and end-to-end benchmarking.

7 EVALUATION

7.1 Evaluation Design

The project evaluation had two layers:

- **Retrieval evaluation:** measure whether the system retrieves the correct evidence chunks.
- **End-to-end evaluation:** compare the full agent against a plain general LLM baseline.

For retrieval evaluation, each benchmark question was paired with a manually curated gold evidence set. A retrieved result counted as relevant only if its returned chunk identifier matched one of the labeled gold chunks. This made the metrics deterministic and reproducible.

For end-to-end evaluation, the comparison involved:

- **Plain LLM baseline:** a general LLM answering the question without course documents.
- **Full agent:** the project system answering through course retrieval and grounded response generation.

Answers were judged on correctness, groundedness, course specificity, and study usefulness.

7.2 Round 1: Controlled Synthetic Benchmark

The first benchmark used a temporary operating-systems mini-course created specifically for evaluation. A small generated document contained four clean sections: processes vs. threads, synchronization, virtual memory, and page replacement. This made it possible to define unambiguous gold chunks.

The main takeaway from Round 1 is that the system’s vector and hybrid retrieval worked perfectly on clean data, while keyword retrieval completely failed on natural-language questions. The full agent beat the plain LLM baseline on every test case, not because it was inherently more knowledgeable, but because it was grounded in the course material.

Table 3: Round 2 retrieval results on the real CS 521 benchmark.

Mode	Hit@5	Recall@5	MRR
Keyword	0.00	0.00	0.00
Vector	0.625	0.5625	0.4792
Hybrid	0.625	0.5625	0.4167

Table 4: Round 2 agent vs. plain LLM results on the real CS 521 benchmark.

System	Corr.	Ground.	Course	Useful
Agent	4.625	4.50	4.50	4.50
Plain LLM	4.125	1.125	1.125	3.875

7.3 Round 2: Real Course Benchmark on CS 521

The second benchmark used the real CS 521 course in the local database. Eight benchmark questions were manually curated across cryptographic primitives, Bitcoin, Ethereum, and consensus. This round is more representative of the system’s actual performance because the source material is noisier and more diverse than the synthetic benchmark.

The agent won 7 out of 8 real-course comparisons. The strongest result was not raw correctness but *groundedness*: the system’s answers were much more likely to align with the actual course materials. At the same time, the real-course benchmark also revealed the system’s main weakness. Retrieval quality dropped substantially compared with the synthetic setting, and keyword retrieval remained nearly unusable on natural-language questions.

7.4 Discussion of Evaluation Results

The evaluation leads to three major conclusions.

First, the system’s main value is *course-grounded assistance*. A strong plain LLM can often give a reasonable generic answer, but it lacks awareness of which explanation is best supported by the course documents. The project consistently improved groundedness and course specificity.

Second, *retrieval is the main bottleneck*. The end-to-end system is only as strong as the evidence it retrieves. The evaluation showed that keyword retrieval fails badly on natural questions and that vector retrieval, while useful, is still imperfect on real materials.

Third, *hybrid retrieval is not yet adding enough over vector retrieval*. On the real benchmark, hybrid did not improve the result because keyword retrieval contributed little useful evidence. This is an important negative finding because it identifies where future engineering effort should go.

8 CHALLENGES AND LESSONS LEARNED

The project involved several practical challenges:

- **Pipeline latency:** topic extraction originally caused noticeable slowdown during upload and deletion, which required restructuring the topic pipeline toward incremental synchronization.

- **Noisy course documents:** PDF lecture materials often contain speaker names, headings, layout artifacts, and fragmented text, which reduce both topic quality and retrieval quality.
- **Grounding vs. fluency:** a general LLM can sound more complete, but a study system should prioritize course alignment rather than generic confidence.
- **System integration:** documents, topics, study plans, and chat had to be linked cleanly so that the final product behaved like one workflow instead of disconnected tools.

These challenges shaped the final design. In particular, the evaluation framework became necessary not just for grading, but for understanding whether the system was failing due to retrieval or due to answer generation.

9 LIMITATIONS AND FUTURE WORK

The system is already usable as a local prototype, but it still has clear limitations.

9.1 Current Limitations

- Keyword retrieval is weak on natural-language questions.
- Real PDF slides can produce noisy chunks and noisy topic candidates.
- Hybrid retrieval currently inherits too much weakness from keyword retrieval.
- The evaluation benchmark is still relatively small.
- Local file storage is sufficient for prototyping but not for broader deployment.

9.2 Future Extensions

Natural next steps include:

- better query rewriting before keyword retrieval
- reranking after initial retrieval
- stronger PDF cleanup and chunking heuristics
- larger multi-course evaluation benchmarks
- optional web deployment for broader student access

10 INDIVIDUAL CONTRIBUTION

This was a solo project. Shengxiang Qi was responsible for the entire project, including idea refinement, system design, backend implementation, frontend implementation, retrieval integration, topic extraction redesign, study-plan workflow, evaluation design, benchmark construction, and final report preparation.

11 CONCLUSION

This project developed a complete retrieval-augmented study assistant for course review. The system goes beyond generic question answering by combining document upload, topic extraction, study planning, topic review, and grounded course chat into one integrated student workflow. The resulting software demonstrates that a course-grounded assistant can be substantially more useful than a plain general LLM for actual study tasks.

The project also shows that useful study support requires more than adding retrieval to a chatbot. The system had to connect ingestion, chunking, topic organization, planning, review, and progress tracking into one coherent workflow so that the user could move

from raw course files to concrete review actions. This integration is the main software contribution of the project. In particular, the topic-centered design helps bridge the gap between document storage and actual studying, while the plan and mastery features make the interface more aligned with learning progress than a plain conversational tool.

At the same time, the evaluation identifies a clear technical boundary. The system is strong when it can retrieve the right course evidence, but its performance falls when retrieval quality drops on noisy real materials. This makes retrieval robustness the main

priority for future improvement. Even so, the current prototype already demonstrates the central claim of the project: a course-grounded, topic-centered study assistant can provide more trustworthy and more actionable support than a general LLM used without structured course context.

A PROJECT REPOSITORY

The source code for this project is available at:

[https://github.com/laiwuqxsx/Study-Planner-RAG_
Based-Agent](https://github.com/laiwuqxsx/Study-Planner-RAG_Based-Agent)